

Entrega # 3

Alumnos:

Juan Sebastián Villa Rodas.
(1027804015)

Miguel Ángel Guarnizo Salazar.
(1038868741)

Tomas Buritica Jaramillo.
(1033179934)

Docente:

Juan Esteban Gómez Tirado.

Materia:

ANÁLISIS Y DISEÑO DE ALGORITMOS.

Año:

2026-1

Tabla de Contenidos

• Portada	pg1.
• Introducción	pg3.
• Solución con Divide y Vencerás	pg3.
• Identificación de subproblemas	pg4.
• Implementación del algoritmo	pg4.
• Fases del algoritmo	pg5.
• Análisis de complejidad	pg6.
• Comparativa final	pg7.
• Tabla comparativa	pg7.
• Análisis de resultados	pg8.
• Gráfica comparativa	pg9.
• Conclusiones	pg10.

INTRODUCCIÓN

En esta última entrega del proyecto se implementó el paradigma de Divide y Vencerás aplicado al problema de planificación de horarios universitarios. Además, se realizó una comparación general entre las cinco técnicas desarrolladas durante todo el proyecto: fuerza bruta, recursividad, greedy, backtracking y divide y vencerás.

El objetivo principal fue analizar cómo cambia el comportamiento de cada algoritmo dependiendo de la estrategia utilizada, comparando aspectos como complejidad, rendimiento y calidad de solución.

1. SOLUCIÓN CON DIVIDE Y VENCERÁS

El paradigma Divide y Vencerás consiste en dividir un problema grande en subproblemas más pequeños, resolverlos de forma independiente y posteriormente combinar sus resultados para obtener una solución final.

En este proyecto, el conjunto de materias se divide recursivamente en dos partes:

- lado izquierdo,
- lado derecho.

Cada subconjunto es procesado por separado utilizando la misma lógica de asignación de horarios.

Finalmente, las soluciones parciales son combinadas validando que no existan conflictos entre profesores, grupos o salones.

1.1 IDENTIFICACIÓN DE SUBPROBLEMAS

El problema de planificación de horarios presenta una estructura que permite dividir el conjunto de materias en grupos más pequeños.

La estrategia utilizada fue:

- dividir el arreglo de materias en dos mitades,
- resolver cada mitad de forma independiente,
- unir los resultados obtenidos.

El caso base ocurre cuando el subconjunto contiene pocas materias, momento en el cual la asignación se realiza directamente.

Esta división permite reducir el tamaño del problema progresivamente y organizar mejor el procesamiento de las asignaciones.

1.2 IMPLEMENTACIÓN DEL ALGORITMO

```

20  ✓ def asignar_materias(materias, salones, horarios):
21
22      asignaciones = []
23
24      for materia in materias:
25
26          asignada = False
27
28          for salon in salones:
29
30              if salon["capacidad"] >= materia["alumnos"]:
31
32                  for horario in horarios:
33
34                      conflicto = hay_conflicto(
35                          asignaciones,
36                          materia["profesor"],
37                          materia["grupo"],
38                          salon["nombre"],
39                          horario
40                      )
41
42                      if conflicto == False:
43
44                          nueva = {
45                              "materia": materia["nombre"],
46                              "profesor": materia["profesor"],
47                              "grupo": materia["grupo"],
48                              "salon": salon["nombre"],
49                              "horario": horario
50                          }
51
52                          asignaciones.append(nueva)
53
54                          asignada = True
55
56                      break

```

La implementación del algoritmo se dividió en dos funciones principales.

Función asignar_materias()

Esta función se encarga de recorrer:

- materias,
- salones,

- horarios.

Para cada materia se busca una combinación válida que:

- no genere conflictos,
- cumpla la capacidad del salón,
- y respete las restricciones establecidas.

Cuando encuentra una asignación válida, esta se almacena en la lista de resultados.

```
64  def divide_y_venceras(materias, salones, horarios):
65
66      # caso base
67      if len(materias) <= 2:
68
69          return asignar_materias(
70              materias,
71              salones,
72              horarios
73          )
74
75      # dividir
76      mitad = len(materias) // 2
77
78      izquierda = materias[:mitad]
79      derecha = materias[mitad:]
80
81      # vencer
82      solucion_izquierda = divide_y_venceras(
83          izquierda,
84          salones,
85          horarios
86      )
87
88      solucion_derecha = divide_y_venceras(
89          derecha,
90          salones,
91          horarios
92      )
93
94      # combinar
95      resultado = solucion_izquierda + solucion_derecha
96
97      return resultado
```

[https://github.com/Miguel-Angel-Salazar/Proyecto_Analisis_Dise-
o/blob/main/2-src/divide_y_venceras.py](https://github.com/Miguel-Angel-Salazar/Proyecto_Analisis_Dise-
o/blob/main/2-src/divide_y_venceras.py)

Función *divide_y_venceras()*

Esta función implementa la lógica principal del paradigma.

El proceso realizado es:

- verificar el caso base,
- dividir las materias en dos subconjuntos,
- resolver cada subconjunto recursivamente,
- combinar las soluciones obtenidas.

1.3 FASES DEL ALGORITMO

FASE 1 — DIVIDIR

El conjunto de materias se divide en dos partes aproximadamente iguales utilizando el punto medio de la lista.

Por ejemplo:

- izquierda → primeras materias,
- derecha → materias restantes.

Esta división permite trabajar con problemas más pequeños y organizados.

FASE 2 — VENCER

Cada subconjunto se resuelve de manera recursiva utilizando la misma estrategia.

Las llamadas recursivas continúan hasta llegar al caso base, donde el conjunto es lo suficientemente pequeño para resolverse directamente.

FASE 3 — COMBINAR

Las asignaciones generadas por cada mitad son unidas en una sola lista final.

Antes de agregar una asignación al resultado final, el algoritmo valida:

- conflictos de horario,
- conflictos de profesor,
- conflictos de grupo,
- conflictos de salón.

De esta manera se garantiza que el horario final siga siendo válido.

1.4 ANÁLISIS DE COMPLEJIDAD

El comportamiento del algoritmo puede representarse mediante la siguiente recurrencia:

$$T(n) = 2T(n/2) + O(n)$$

La división recursiva del problema permite reducir progresivamente el tamaño de entrada.

Teóricamente, utilizando el Teorema Maestro, este comportamiento se aproxima a:

$$O(n \log n)$$

Sin embargo, debido a las validaciones internas de conflictos y asignaciones, el comportamiento práctico puede variar dependiendo de la cantidad de horarios, salones y materias disponibles.

La complejidad espacial es aproximadamente:

$$O(n)$$

ya que las asignaciones y llamadas recursivas deben almacenarse temporalmente.

2. COMPARATIVA FINAL

TABLA COMPARATIVA

Técnica	Complejidad Temporal	Complejidad Espacial	¿Solución óptima?
Fuerza Bruta	$O(n^2)$	$O(n)$	No
Recursividad	$O(n^2)$	$O(n)$	No
Greedy	$O(n^2)$	$O(n)$	No siempre
Backtracking	$O(b^n)$	$O(n)$	Puede encontrar mejores soluciones
Divide y Vencerás	$O(n \log n)$ aprox.	$O(n)$	Parcial

ANÁLISIS DE RESULTADOS

Cada técnica presentó ventajas y desventajas dependiendo del objetivo buscado.

La solución por fuerza bruta fue útil para comprender el problema, aunque su rendimiento disminuye cuando aumenta el tamaño de entrada.

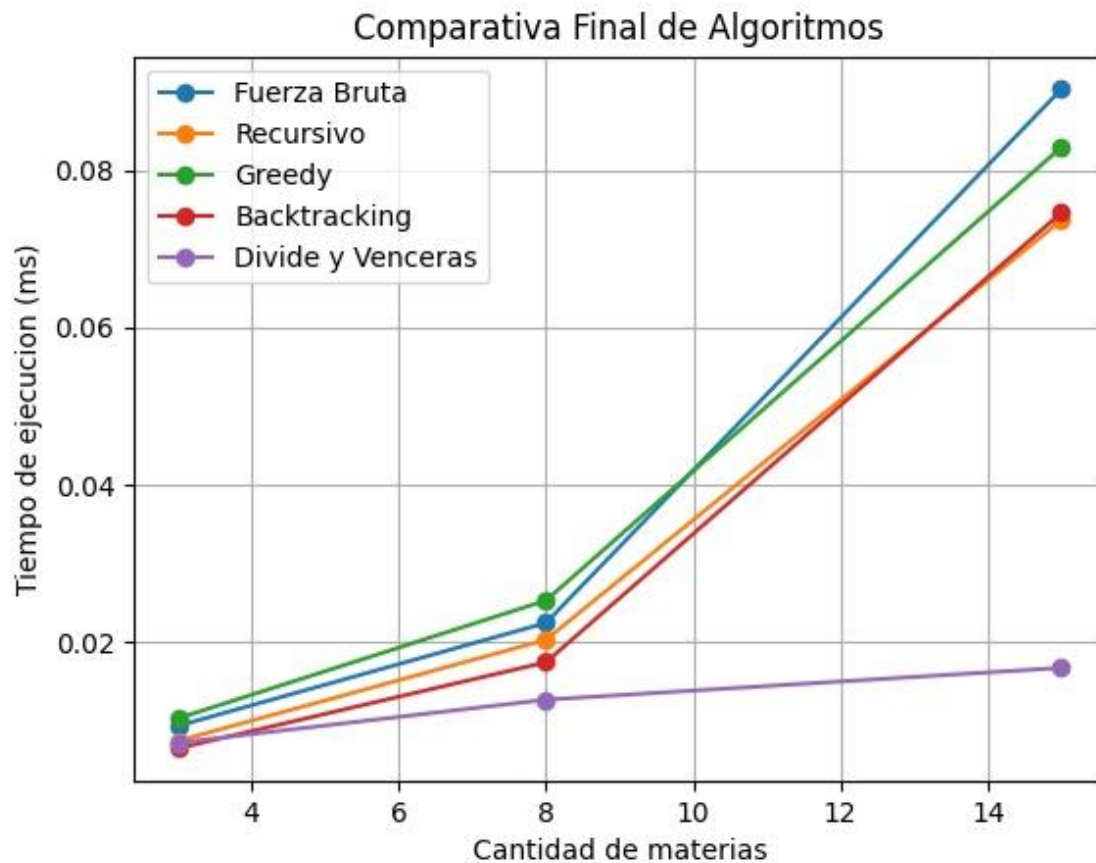
La solución recursiva mantuvo un comportamiento similar, pero permitió una estructura de código más organizada.

Greedy destacó por su rapidez, ya que toma decisiones inmediatas sin explorar demasiadas posibilidades. Sin embargo, esto puede ocasionar que no siempre encuentre la mejor distribución posible.

Backtracking fue el algoritmo más completo en exploración de soluciones, ya que prueba diferentes combinaciones antes de tomar una decisión definitiva. Aun así, esto provoca un mayor costo computacional.

Finalmente, Divide y Vencerás permitió organizar el problema en subproblemas más pequeños, obteniendo un comportamiento eficiente y una estructura más clara para resolver el sistema de horarios.

GRÁFICA COMPARATIVA



[https://github.com/Miguel-Angel-Salazar/Proyecto Analisis Dise-
o/blob/main/4-graficas/comparativa_final.png](https://github.com/Miguel-Angel-Salazar/Proyecto_Analisis_Dise-
o/blob/main/4-graficas/comparativa_final.png)

EXPLICACIÓN DE LA GRÁFICA

La gráfica muestra el crecimiento del tiempo de ejecución de los cinco algoritmos conforme aumenta la cantidad de materias.

Los algoritmos fuerza bruta y recursivo presentan comportamientos similares debido a que realizan recorridos parecidos.

Greedy mantiene tiempos bajos gracias a su estrategia de decisiones rápidas.

Backtracking aumenta considerablemente su costo computacional debido a la exploración de múltiples combinaciones posibles.

Por último, Divide y Vencerás presenta un comportamiento más estable gracias a la división recursiva del problema en subconjuntos más pequeños.

¿QUÉ TÉCNICA FUE LA MÁS ADECUADA?

Para el problema planteado, la técnica greedy resultó ser una de las más rápidas y sencillas de implementar.

Sin embargo, backtracking permitió explorar soluciones más completas cuando el objetivo era encontrar asignaciones más válidas considerando distintas posibilidades.

Por otro lado, Divide y Vencerás mostró un comportamiento eficiente y organizado, especialmente cuando aumenta el tamaño del problema.

Cada técnica resultó útil dependiendo del objetivo buscado:

- rapidez,
- simplicidad,
- exploración de soluciones,
- o escalabilidad.

3. CONCLUSIONES

Durante el desarrollo del proyecto fue posible aplicar distintos paradigmas de programación al mismo problema, permitiendo comprender cómo cambia el comportamiento de un algoritmo dependiendo de la estrategia utilizada.

La solución por fuerza bruta ayudó a entender el funcionamiento básico del problema, mientras que la recursividad permitió organizar mejor la implementación mediante llamadas recursivas.

Greedy mostró cómo una solución rápida puede ser eficiente en muchos casos, aunque no siempre garantiza la mejor asignación posible.

Backtracking permitió explorar más soluciones mediante retrocesos y validación de restricciones, aunque con un mayor costo computacional.

Finalmente, Divide y Vencerás permitió dividir el problema en partes más pequeñas para resolverlo de forma más organizada y eficiente.

En escenarios reales:

- greedy podría utilizarse en sistemas donde se necesiten respuestas rápidas,
- backtracking en problemas donde la precisión sea más importante,
- divide y vencerás en sistemas con grandes cantidades de datos,
- y fuerza bruta para validaciones o pruebas pequeñas.

Entre las principales limitaciones encontradas estuvieron:

- el aumento del tiempo de ejecución en algunos algoritmos,
- la dificultad de manejar muchas restricciones simultáneamente,
- y la necesidad de optimizar aún más las soluciones para casos grandes.